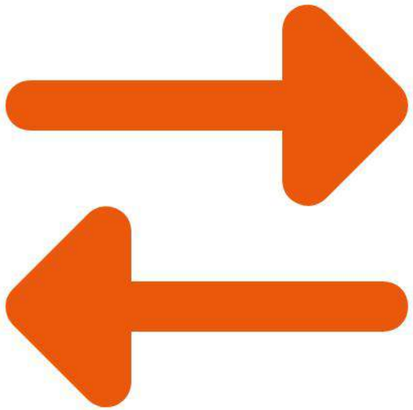
PYTHON TUTORIAL

Reverse Line Generator

A guide to string manipulation and file processing in Python.

Prepared for Python Learners

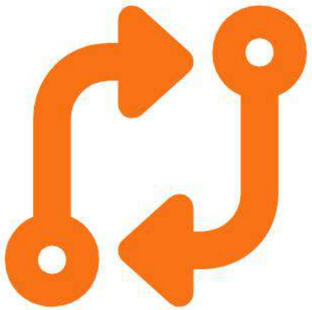


The Challenge

Goal

Read a file `lines.txt` containing one word per line, reverse each word, and save the result to `reversed.txt`.

- **Reversal:** 'apple' becomes 'elppa'.
- **Formatting:** Maintain one word per line.
- **Safety:** Execute without errors.



Input vs. Output

lines.txt (Input)

```
apple
banana
cherry
```

Reverse

reversed.txt (Output)

```
elppa
ananab
yrrehc
```

Key Concept: String Slicing

The `[::-1]` Idiom

Python offers a powerful way to slice sequences using the syntax `[start:stop:step]`.

By setting the **step** to `-1`, we tell Python to traverse the string backwards.

- `word = "Python"`
- `word[::-1]` → `"nohtyP"`

start:stop:-1

Solution 1: Iterative Approach

The Logic

We process the file line by line. This is crucial for memory management if the file is huge.

Warning: The Newline Trap

Lines from files usually end with `\n`. If you reverse `"apple\n"`, you get `"\nelppa"`—the newline moves to the front!

Fix: Strip the whitespace *before* reversing.

The Trap

`"apple\n"[::-1]`

↓

`"\nelppa"`

(Broken formatting)

Flowchart: Iterative Loop

Open Files

Read (in) & Write (out)

Strip

Remove `\n`: `line.strip()`

Reverse

Flip it: `word[::-1]`

Write

Add new `\n` & save

Repeat

Until end of file

Code: Iterative Solution

```
def reverse_lines_iterative():
    try:
        with open('lines.txt', 'r') as infile, \
             open('reversed.txt', 'w') as outfile:

            for line in infile:
                # 1. Clean (remove \n)
                clean_word = line.strip()

                # 2. Reverse
                reversed_word = clean_word[::-1]

                # 3. Write with NEW newline
                outfile.write(f"{reversed_word}\n")

    except FileNotFoundError:
        print("Input file not found.")
```

Code Breakdown

- `line.strip()`: Essential to remove the original newline character so it doesn't get reversed to the front.
- `[::-1]`: The Pythonic reverser.
- `write( ... \n)`: We manually add a fresh newline at the correct position (the end).

Solution 2: Bulk Processing

The "Read, Split, Reverse, Join"

If the file is small, we can read it all at once. This allows for very concise code.

- **read():** Gets the whole file.
- **splitlines():** Breaks it into a list of strings, *automatically removing newlines*.
- **List Comp:** Reverses every item in the list.
- **join():** Rebuilds the file string.

List Operations

Flowchart: Bulk Method

1. Ingest

`f.read().splitlines()`

Load file & remove newlines instantly.

2. Process

`[x[::-1] for x in lines]`

Reverse every item in the list.

3. Export

`"\n".join(list)`

Combine & write to disk.

Code: Bulk Solution

```python
def reverse_lines_bulk():
    try:
        with open('lines.txt', 'r') as f:
            # splitlines() removes \n automatically
            lines = f.read().splitlines()

        # Reverse each line in one go
        reversed_lines = [line[::-1] for line in lines]

        with open('reversed.txt', 'w') as f:
            # Join them back together
            f.write("\n".join(reversed_lines))

    except FileNotFoundError:
        print("File missing.")
```

Code Breakdown

- `splitlines()`: Cleaner than `readlines()` because it handles newline removal for us.
- `join()`: Ensures we don't have an extra empty line at the very end of the file.
- **Conciseness:** Logic is separated from File I/O.

Comparing Approaches

Feature	Solution 1 (Iterative)	Solution 2 (Bulk)
Newline Handling	Manual `strip()` required	Auto via `splitlines()`
Memory	**Low** (One line at a time)	High (Loads full file)
Formatting	Adds trailing newline	**Clean** (No trailing newline)
Complexity	Beginner Friendly	Intermediate / Pythonic

Why Formatting Matters

The Trailing Newline

When writing files, Solution 1 typically ends with:

`elppa\nananab\nyrrehc\n`

Solution 2 (Join) typically ends with:

`elppa\nananab\nyrrehc`

Both are often acceptable, but Solution 2 is often preferred for data exchange files to avoid empty trailing records.

Visual Check

```
1. elppa
2. ananab
3. yrrehc
4.   <-- Solution 1 creates this
```